

# Dokumentacja projektu: Paper Soccer (Piłkarzyki na kartce)

Autorzy: Szymon Rafałowski, Wiktor Sasnał

Politechnika Krakowska, Wydział Informatyki i Telekomunikacji

## 1. Opis projektu

Cyfrowa wersja szkolnej gry "Piłkarzyki na kartce". Aplikacja desktopowa w języku Java z interfejsem w bibliotece Swing. Dostępne tryby: gra dwóch osób na jednym komputerze (hot-seat) oraz gra przeciwko komputerowi z trzema poziomami trudności.

## 2. Zasady gry

Piłka zaczyna na środku boiska zbudowanego z siatki węzłów. Gracze na przemian rysują odcinki o długości jednego pola (w pionie, poziomie lub na ukos), tworząc tor piłki.

- Nie można rysować po odcinku już narysowanym ani po bandzie boiska.
- Wejście na węzeł już odwiedzony lub na bandę daje dodatkowy ruch (odbicie). Tura zostaje przy tym samym graczem.
- Wejście na nowy węzeł kończy turę i oddaje ruch przeciwnikowi.
- Wprowadzenie piłki do bramki przeciwnika kończy grę zwycięstwem.
- Gracz, który nie ma żadnego dozwolonego ruchu, przegrywa (blokada).
- Do bramki można wejść tylko przez jej otwór.

## 3. Uruchomienie

Wymagania: Java 17 lub nowsza.

Kompilacja i uruchomienie bez Mavena:

```
javac -d out src/main/java/pl/edu/pk/papersoccer/*.java
java -cp out pl.edu.pk.papersoccer.PaperSoccerApp
```

Przez Mavena:

```
mvn -q compile exec:java
```

Testy jednostkowe (wymagają Mavena):

```
mvn test
```

## 4. Struktura projektu

Kod znajduje się w pakiecie `pl.edu.pk.papersoccer`, układ katalogów jest zgodny z Mavenem (`src/main/java`, `src/test/java`).

- `PaperSoccerApp` : punkt wejścia. Tworzy okno i przełącza ekrany (menu, wybór poziomu, gra) za pomocą `CardLayout`.
- `MenuPanel` : menu główne, wybór trybu gry.
- `DifficultyPanel` : wybór poziomu trudności gry z komputerem.
- `GamePanel` : rysowanie boiska i obsługa myszy. Animuje ruchy komputera przy użyciu timera.
- `GameLogic` : stan gry i reguły. Plansza, walidacja ruchu, zmiana tur, wykrywanie gola i blokady.
- `Line` : pojedynczy odcinek między dwoma węzłami.
- `Difficulty` : poziomy trudności wraz z głębokością przeszukiwania.
- `MinimaxAi` : logika przeciwnika komputerowego.

Logika gry jest oddzielona od interfejsu. `GamePanel` tylko rysuje i zbiera kliknięcia, a wszystkie reguły są w `GameLogic`. Dzięki temu te same reguły wykorzystuje gracz, mechanizm wykrywania końca gry oraz komputer.

## 5. Plansza i walidacja ruchu

Boisko to siatka węzłów o współrzędnych  $x$  od 0 do 8 oraz  $y$  od 0 do 10. Piłka startuje na środku, w punkcie (4, 5). Bramki znajdują się przy  $x$  od 3 do 5, u góry ( $y$  równe 0) oraz u dołu ( $y$  równe 10).

Ruch jest poprawny, gdy spełnia wszystkie warunki: prowadzi do sąsiedniego węzła (jedno pole w jednym z ośmiu kierunków), nie powiela istniejącego odcinka ani bandy, nie wychodzi poza planszę oraz wchodzi do bramki wyłącznie przez jej otwór. Krzyżowanie odcinków na ukos jest dozwolone.

Po wykonaniu ruchu sprawdzane jest po kolei: czy padł gol (piłka trafiła do wiersza bramki), czy nastąpiło odbicie (węzeł był już odwiedzony, więc tura zostaje), oraz czy gracz na ruchu nie został zablokowany (brak dozwolonych ruchów oznacza przegraną).

## 6. Przeciwnik komputerowy (algorytm minimax)

Komputer wybiera ruch algorytmem minimax z odcięciami alfa-beta.

- Komputer maksymalizuje swój wynik, zakładając, że człowiek gra najlepiej, czyli minimalizuje wynik komputera.
- Gracz znajdujący się na ruchu jest w każdym stanie odczytywany z pola tury, a nie zakładany na sztywno. Dzięki temu odbicia (gdy ten sam gracz rusza ponownie) są obsługiwane w naturalny sposób.
- Stany końcowe oceniane są po położeniu piłki: gol komputera daje wynik dodatni, gol człowieka ujemny, a blokada niezależnie od tego, kto jest na ruchu. Szybsza wygrana jest premiowana.
- Gdy w zasięgu głębokości nie ma rozstrzygnięcia, pozycja oceniana jest heurystyką. Im mniej ruchów dzieli piłkę od bramki przeciwnika, tym lepiej. Dodatkowo brana jest pod uwagę liczba dostępnych ruchów oraz możliwość odbicia.
- Przy równej ocenie wybierany jest ruch, który stawia piłkę bliżej bramki przeciwnika.
- Kolejne ruchy są symulowane na kopii stanu gry, więc algorytm korzysta z tych samych reguł co właściwa rozgrywka.

Poziomy trudności różnią się wyłącznie głębokością przeszukiwania:

- Łatwy: 2 ruchy w przód.
- Średni: 4 ruchy w przód.
- Trudny: 7 ruchów w przód.

Wszystkie poziomy grają deterministycznie, bez losowości.

## 7. Testy jednostkowe

Projekt zawiera 13 testów napisanych w JUnit 5.

- Reguły gry: pozycja startowa, ruch prosty i na ukos, ruch za daleki, brak powielania odcinka, zmiana tury, dodatkowy ruch po odbiciu, zakaz wejścia do bramki zza słupka.
- Komputer: zwracany ruch jest zawsze poprawny, komputer strzela gola gdy ma taką możliwość, wybór jest deterministyczny, rozegranie całej partii nie powoduje nielegalnego ruchu, a głębokości poziomów są zgodne z założeniami.

## 8. Użyte narzędzia

- Java 17 (Swing, AWT)

- Maven
- JUnit 5
- Git

## 9. Możliwe rozszerzenia

- Gra sieciowa dla dwóch osób na różnych komputerach. Przewidziana jako opcja rozwojowa, nie weszła do tej wersji.
- Głębsze przeszukiwanie lub rozbudowana heurystyka komputera.